

接口定义文档 v0.2

本文替代 B3_AB接口定义文档_v0.1。 schema_version 为 2.0。机器可读定义见同目录 ab-job-contract.schema.json。校验某个 JSON 时，使用下表对应的 \$defs 条目。对象中的未知字段必须忽略；枚举之外的值必须拒绝。字段表、本节规则与 JSON Schema 同时生效。

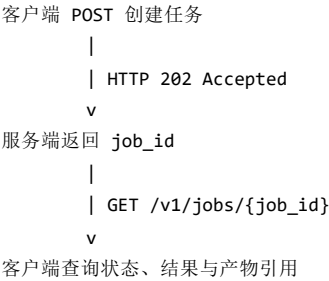
相对 v0.1，本版只改联调前必须对齐的契约：

- 1. 补上 REPAIR 之后由流程发起方产生新 commit 的交接。
- 2. 固定 Artifact URI 到共享目录的映射，跨组产物的 sha256 必填。
- 3. 全链路使用同一种 image_ref。
- 4. 四种请求复用 EnvironmentRef 与同一套 build 字段；创建回执与查询 Job 分开定义。
- 5. 增加 JSON Schema，并因字段变更把 schema_version 升到 2.0。
- 6. 写明 DRAFT 与 REPAIR 何时为 SUCCEEDED。

本版不改：幂等键的比较细节、finding_id 的派生算法、取消接口、轮询间隔、鉴权，以及 actual.json / declared.json 的内部字段。依赖图文件只通过 ArtifactRef 传递。

1. 总体接口风格

双方统一采用：异步 Job + JSON 请求/响应 + Artifact 引用。



1.1 通用请求头

```
Content-Type: application/json
Accept: application/json
Idempotency-Key: 2d6bf47d-d692-4ef0-b82c-e23dd3a375da
```

- 1. 所有创建任务的 POST 请求必须携带 Idempotency-Key。
- 2. 同一服务收到相同幂等键和相同请求体时，应返回同一个 job_id，不得重复创建任务。
- 3. 同一服务收到相同幂等键但请求体不同时，应返回 409 Conflict。
- 4. job_id 由接收任务的服务端生成，匹配 job-[a-z0-9]+(-[a-z0-9]+)*。
- 5. trace_id 由流程发起方提供，并在同一次完整流程中保持不变。

1.2 参考端点

操作	方法与端点	实现方	成功状态码
生成构建环境	POST /v1/dockerfile-jobs	B	202 Accepted
全量检测	POST /v1/full-check-jobs	A	202 Accepted
增量检测	POST /v1/incremental-check-jobs	A	202 Accepted
修复缺失依赖	POST /v1/repair-jobs	B	202 Accepted

操作	方法与端点	实现方	成功状态码
查询任务	GET /v1/jobs/{job_id}	接收该任务的服务	200 OK

请求体中的 `job_type` 必须与端点一致。不一致时返回 `422`，错误码 `VALIDATION_1001`。

端点名称来自课程 PPT 的参考方案。如配对组修改端点名称，必须保持请求与响应语义不变，并在 ADR 中记录。

1.3 一次完整流程

流程发起方是四类 POST 的客户端。一次 `trace_id` 按下面的顺序交接：

1. POST `/v1/dockerfile-jobs`。成功后得到 `Dockerfile` 与 `image_ref`。
2. POST `/v1/full-check-jobs`。环境使用上一步的 `EnvironmentRef`，源码使用当前 `repository.commit`。
3. POST `/v1/repair-jobs`。只选择 `MISSING` finding。成功后得到可 `git apply` 的 `patch`，以及 `base_commit`。
4. 流程发起方在仓库根目录对该 `base_commit` 执行 `git apply`，并产生新的完整 `commit SHA`。`REPAIR` 不推送，也不返回新 `commit`。
5. 用新 `SHA` 调用 POST `/v1/incremental-check-jobs` 或再一次全量检测。`baseline.commit` 等于修补前的 `base_commit`，`repository.commit` 等于新 `SHA`。

`image_ref` 表示构建环境，不随 `Makefile` 补丁自动变化。`Dockerfile` 与镜像都未变化时，后续检测继续使用同一次 `DRAFT` 的 `EnvironmentRef`。只有构建环境本身需要更新时，才对新 `commit` 重新调用 `DRAFT`。

2. 统一 Job 模型

2.1 查询到的 Job

GET `/v1/jobs/{job_id}` 返回完整 Job。创建请求被受理时不返回这个对象，只返回 §3.1 的回执。

```
{
  "schema_version": "2.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "RUNNING",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": null
  },
  "input": {},
  "output": null,
  "error": null
}
```

2.2 公共字段

字段	类型	创建回执 202	查询 200	说明
schema_version	string	必填	必填	本版固定为 2.0
job_id	string	必填	必填	服务端生成
trace_id	string	必填	必填	创建请求必填，响应原样返回
job_type	enum	必填	必填	DRAFT、FULL_CHECK、INCREMENTAL_CHECK、REPAIR
status	enum	必填，且为 QUEUED	必填	当前状态

字段	类型	创建回执 202	查询 200	说明
execution	object	只要求 created_at	created_at 、 started_at 、 finished_at 都必填	时间；未发生则为 null
input	object	不得出现	必填	创建请求中的 input 原样保留
output	object 或 null	不得出现	必填	成功结果；未成功时为 null
error	object 或 null	不得出现	必填	执行异常；无异常时为 null

创建回执的 status 必须是 QUEUED 。之后是否进入 RUNNING ，只以查询结果为准。

2.3 job_type

DRAFT 、 FULL_CHECK 、 INCREMENTAL_CHECK 、 REPAIR 。

2.4 status 与成功条件

QUEUED -> RUNNING -> SUCCEEDED
-> FAILED
-> TIMED_OUT
-> CANCELLED

状态	含义	终态	output	error
QUEUED	已受理，尚未开始	否	null	null
RUNNING	正在执行	否	null	null
SUCCEEDED	按本节成功条件完成	是	对象	null
FAILED	未能按成功条件完成	是	null	对象
TIMED_OUT	超过时间限制	是	null	对象
CANCELLED	任务被取消	是	null	对象

成功条件：

任务	SUCCEEDED	FAILED 且 output 为 null
FULL_CHECK 、 INCREMENTAL_CHECK	检测工具正常结束。发现 MISSING 或 REDUNDANT 仍算成功	工具没有正常结束，例如 ANALYSIS_5001
DRAFT	产出 Dockerfile 与镜像，且 build_result 与 test_result 的 exit_code 都为 0	镜像构建失败用 ENV_3002 。达到 max_iterations 仍未通过测试用 DRAFT_3001 。不得交付 image
REPAIR	候选 patch 已通过构建和测试，且重检报告不再包含所选 finding_id	候选无效用 REPAIR_3001 。不得把该 patch 放进 output

2.5 execution

时间使用 UTC，RFC 3339，且以 z 结尾。尚未开始时 started_at 为 null 。尚未结束时 finished_at 为 null 。

3. 创建任务与查询任务

3.1 创建回执

对应 Schema：CreateReceipt。

```
HTTP/1.1 202 Accepted
Content-Type: application/json
Location: /v1/jobs/job-full-001
```

```
{
  "schema_version": "2.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "QUEUED",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z"
  }
}
```

3.2 查询运行中的任务

对应 Schema：Job。

```
{
  "schema_version": "2.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "RUNNING",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": null
  },
  "input": {},
  "output": null,
  "error": null
}
```

3.3 查询成功任务

status 为 SUCCEEDED 时，output 使用该 job_type 的成功输出，error 为 null。

3.4 查询执行失败任务

```
{
  "schema_version": "2.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "FAILED",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": "2026-09-22T12:00:41Z"
  },
  "input": {},
  "output": null,
  "error": {
    "code": "ANALYSIS_5001",
    "message": "Analyzer failed",
    "details": {
      "log_uri": "artifact://pair03/job-full-001/analyzer.log"
    }
  }
}
```

4. Artifact 产物引用

Job 中只保存元数据和产物引用，不直接嵌入大日志、依赖图或 Patch。

4.1 ArtifactRef

跨组传递的每个 `ArtifactRef` 都必须带 `sha256`。依赖源码的产物必须带完整 `commit`。依赖构建配置的产物必须带 `configuration_id`。

[illegible]

字段	必填	说明
artifact_id	是	产物编号
type	是	见 §4.2
uri	是	文件产物使用 artifact:// ； CONTAINER_IMAGE 的 uri 等于 image_ref
media_type	是	MIME 类型
producer_job_id	是	生成该产物的 Job
commit	依赖源码时必填	完整提交 SHA
configuration_id	依赖构建配置时必填	构建配置
sha256	是	小写十六进制 64 位。 CONTAINER_IMAGE 的这个值必须等于 uri 里 @sha256: 后面的 digest

4.2 产物类型

DOCKERFILE 、 CONTAINER_IMAGE 、 BUILD_LOG 、 TEST_LOG 、 ANALYZER_LOG 、 ACTUAL_GRAPH 、 DECLARED_GRAPH 、 ERROR_REPORT 、 PATCH 、 REPAIR_REPORT 。

4.3 URI 与读取

文件产物的 URI 固定为：

```
artifact://{pair_id}/{job_id}/{file_name}
```

pair_id 匹配 [a-z0-9]+(-[a-z0-9]+)* 。 job_id 必须等于 producer_job_id 。 file_name 只含字母、数字、点、下划线和短横线，不含路径分隔符。

联调时映射到仓库内共享路径：

```
artifacts/{pair_id}/{job_id}/{file_name}
```

因此 artifact://pair03/job-full-001/error-report.json 就是 artifacts/pair03/job-full-001/error-report.json 。下游必须能读到该文件，并核对 sha256 、 commit 与 configuration_id 。禁止交接本机绝对路径。

5. Repository、Build 与 Environment

5.1 RepositoryRef

```
{
  "url": "https://example.com/pair03/demo-project.git",
  "commit": "0123456789abcdef0123456789abcdef01234567",
  "project_root": "."
}
```

commit 必须是完整 SHA。 project_root 相对仓库根目录；仓库根目录写 . 。

5.2 build

四种请求使用同一组字段名。某一类请求不使用的字段不出现。

字段	DRAFT	FULL_CHECK / INCREMENTAL_CHECK	REPAIR
configuration_id	不出现	必填	必填
clean_command	不出现	必填	必填
build_command	必填	必填	必填
test_command	必填	不出现	必填

v0.1 的 verify_command 改名为 test_command 。REPAIR 不再接收 recheck_command 。

5.3 EnvironmentRef

检测与修复请求中的 environment 都使用这个对象。image_ref 与对应 CONTAINER_IMAGE 的 uri 是同一字符串，形式为 {name}@sha256:{digest} 。 dockerfile.producer_job_id 必须与产出该镜像的 DRAFT 相同。


```
{
  "schema_version": "2.0",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "input": {
    "repository": {
      "url": "https://example.com/pair03/demo-project.git",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "project_root": "."
    },
    "environment": {
      "dockerfile": {
        "artifact_id": "dockerfile-001",
        "type": "DOCKERFILE",
        "uri": "artifact://pair03/job-draft-001/Dockerfile",
        "media_type": "text/plain",
        "producer_job_id": "job-draft-001",
        "commit": "0123456789abcdef0123456789abcdef01234567",
        "sha256": "111111111111111111111111111111111111111111111111111111111111111111"
      },
      "image_ref": "pair03/demo-project@sha256:222222222222222222222222222222222222222222222222222222222222222222"
    },
    "build": {
      "configuration_id": "cc-MODE0",
      "clean_command": "make clean",
      "build_command": "make"
    }
  }
}
```

7.2 成功输出

[illegible]

8. INCREMENTAL_CHECK 接口 (A 组)

Schema: 请求 IncrementalCreateRequest , 成功输出 IncrementalSuccessOutput 。

8.1 创建请求

POST /v1/incremental-check-jobs

下面示例中的 `bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb` 是流程发起方应用 `job-repair-001` 的 `patch` 之后产生的新 `commit`。构建环境仍使用 `job-draft-001`。

8.2 成功输出

```
{
  "current_error_report": {
    "artifact_id": "error-report-c1",
    "type": "ERROR_REPORT",
    "uri": "artifact://pair03/job-incremental-c1/error-report.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0",
    "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
  },
  "finding_changes": {
    "introduced": [],
    "resolved": ["finding-md-001"],
    "unchanged": []
  },
  "updated_actual_graph": {
    "artifact_id": "actual-graph-c1",
    "type": "ACTUAL_GRAPH",
    "uri": "artifact://pair03/job-incremental-c1/actual.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0",
    "sha256": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb"
  },
  "updated_declared_graph": {
    "artifact_id": "declared-graph-c1",
    "type": "DECLARED_GRAPH",
    "uri": "artifact://pair03/job-incremental-c1/declared.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0",
    "sha256": "ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc"
  }
}
```

9. ERROR_REPORT 与 Finding

Schema：ErrorReport。报告文件的 schema_version 也是 2.0。

9.1 文件结构

```
{
  "schema_version": "2.0",
  "report_type": "ERROR_REPORT",
  "producer_job_id": "job-full-001",
  "repository": {
    "url": "https://example.com/pair03/demo-project.git",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "configuration_id": "cc-MODE0",
  "findings": [
    {
      "finding_id": "finding-md-001",
      "type": "MISSING",
      "target": "main.o",
      "dependency": "config.h",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "location": {
        "file": "Makefile",
        "line": 12,
        "column": 1
      },
      "evidence": {
        "summary": "main.o uses config.h but the dependency is not declared",
        "process": "cc -c main.c -o main.o",
        "observed_file": "config.h",
        "declaration_file": "Makefile"
      },
      "detector": "BUILD_CHECKER"
    }
  ]
}
```

9.2 Finding 字段

字段	必填	说明
finding_id	是	报告内稳定且唯一
type	是	MISSING 或 REDUNDANT
target	是	被检测的构建目标
dependency	是	缺失或冗余的依赖项
commit	是	完整提交 SHA
location.file	是	仓库相对路径
location.line	是	从 1 开始
location.column	否	从 1 开始
evidence	是	支持该结论的证据
detector	是	BUILD_CHECKER 、 E_CHECKER 或 INSTRUCTOR_ORACLE

MISSING 表示实际需要但声明缺失。 REDUNDANT 表示已声明但当前配置没有使用。

MDFixer 只消费 MISSING 。报告同时包含两类 finding 时，REPAIR 请求必须用 finding_ids 明确选择 MISSING 。

10. REPAIR 接口 (B 组)

Schema: 请求 RepairCreateRequest , 成功输出 RepairSuccessOutput 。

10.1 创建请求

POST /v1/repair-jobs

[illegible]

接收前校验:

1. `repository.commit` 必须等于 `error_report.commit`。
2. 报告正文中的 `repository.commit` 也必须与二者一致。
3. `finding_ids` 中的每一项必须存在于报告中。
4. 被选择的 Finding 必须全部为 `MISSING`。
5. `build_file` 必须使用仓库相对路径。
6. `build.configuration_id` 必须等于错误报告中的 `configuration_id`。

重检使用与 FULL_CHECK 相同的 finding 规则和同一个 configuration_id。请求中不再包含 recheck_command。

10.2 成功输出

`patch.commit` 与 `base_commit` 都是修补前的提交。`patch` 是针对该提交的 unified diff, `media_type` 为 `text/x-diff`, 在仓库根目录可以被 `git apply` 接受。

[illegible]

候选无效时任务为 FAILED，error.code 为 REPAIR_3001，output 为 null。

11. 错误模型

11.1 任务受理之后的错误

系统错误写入 `job.error`。

错误码	含义	终态
ENV_3002	镜像构建失败	FAILED
DRAFT_3001	DRAFT 达到 <code>max_iterations</code> 仍未通过测试	FAILED
REPAIR_3001	修复候选未通过构建、测试或重检	FAILED
EXEC_4002	任务超时	TIMED_OUT
ANALYSIS_5001	分析器失败	FAILED

DRAFT_3001 与 REPAIR_3001 是本版为成功条件补上的任务错误码。课程材料中的 ENV_3002、EXEC_4002、ANALYSIS_5001 保持不变。

11.2 请求拒绝

请求在创建 Job 前校验失败时，不生成 `job_id`，直接返回 HTTP 4xx。Schema：`HttpError`。

```
{
  "schema_version": "2.0",
  "error": {
    "code": "VERSION_2001",
    "message": "Error report commit does not match repository commit",
    "field": "input.error_report.commit"
  }
}
```

HTTP 状态	错误码	触发条件
400 Bad Request	REQUEST_1000	JSON 无法解析
422 Unprocessable Content	VALIDATION_1001	缺少必填字段、字段格式错误，或 <code>job_type</code> 与端点不一致
422 Unprocessable Content	BASELINE_2001	增量检测缺少 <code>baseline</code>
422 Unprocessable Content	BASELINE_2002	<code>baseline</code> 提交或配置不匹配
422 Unprocessable Content	REPAIR_2001	REPAIR 选择了非 <code>MISSING</code> Finding
409 Conflict	VERSION_2001	报告 <code>commit</code> 与源码 <code>commit</code> 不一致
409 Conflict	IDEMPOTENCY_2002	相同幂等键对应不同请求体
404 Not Found	JOB_2004	<code>job_id</code> 不存在

12. 版本兼容性

`schema_version` 现为 `2.0`。服务端和客户端必须忽略未知字段。未知枚举值必须拒绝。

可兼容变更：新增可选字段；新增不影响既有语义的产物元数据。

破坏性变更：删除或重命名现有字段；改变字段语义或类型；改变状态、任务类型或 Finding 类型枚举；把可选字段改为必填。发生破坏性变更时，必须提升 `schema_version` 的主版本，并同步更新本文件、`ab-job-contract.schema.json`、示例和 ADR。

13. 最小验收用例

编号	用例	预期结果
T01	提交四类合法创建请求	均返回 202、 job_id ， 且回执符合 CreateReceipt
T02	将 job_type 改为 ABC	返回 422 VALIDATION_1001
T03	删除增量请求的 baseline	返回 422 BASELINE_2001
T04	令 baseline commit 与 base commit 不一致	返回 422 BASELINE_2002
T05	REPAIR 选择 REDUNDANT Finding	返回 422 REPAIR_2001
T06	令报告 commit 与仓库 commit 不一致	返回 409 VERSION_2001
T07	检测正常完成并发现 MD	Job 为 SUCCEEDED ， MD 写入 findings
T08	分析器崩溃	Job 为 FAILED ， 错误为 ANALYSIS_5001 ， output 为 null
T09	下游读取 Artifact URI	路径第二段等于 job_id ， 文件可读取， sha256 与字节一致， commit / configuration_id 与引用一致
T10	相同幂等键重复提交同一请求	返回同一个 job_id ， 不重复执行
T11	DRAFT 达到 max_iterations 仍未通过测试	Job 为 FAILED ， error.code 为 DRAFT_3001 ， output 为 null
T12	REPAIR 重检仍包含所选 finding	Job 为 FAILED ， error.code 为 REPAIR_3001 ， output 为 null

14. Schema 对照

载荷	\$defs
创建回执	CreateReceipt
查询 Job	Job
DRAFT 请求 / 成功输出	DraftCreateRequest / DraftSuccessOutput
FULL_CHECK 请求 / 成功输出	FullCheckCreateRequest / FullCheckSuccessOutput
INCREMENTAL_CHECK 请求 / 成功输出	IncrementalCreateRequest / IncrementalSuccessOutput
REPAIR 请求 / 成功输出	RepairCreateRequest / RepairSuccessOutput
错误报告文件	ErrorReport
创建前拒绝	HttpError